

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Activity / Exp No.: Project-Based Learning (PBL) Part 1

Student Name: J.Srikar

Roll No.: A24126552083

Date: 21-09-2026

1. TITLE

Project-Based Learning (PBL) part 1

Interactive Document Object Model (DOM) Manipulation with Dynamic Content, Style, and Attribute Modification

2. AIM

To design and implement a responsive interactive web application using HTML5, CSS3, and vanilla JavaScript demonstrating dynamic Document Object Model (DOM) manipulation, including runtime text content alteration, dynamic CSS style transformation, HTML element attribute reconfiguration, and state restoration via reset functionality.

3. OBJECTIVE

- To construct a modern, visually engaging card layout using CSS linear gradients, flexbox centering, rounded borders, and drop shadows.
- To target and retrieve specific HTML DOM tree nodes using `document.getElementById()`.
- To dynamically alter inner textual content of elements (`textContent`) without re-parsing HTML trees.
- To modify inline CSS style properties (`style.color`, `style.backgroundColor`, `style.fontSize`, `style.borderRadius`, `style.borderColor`) dynamically.
- To manipulate HTML element attributes (`src`, `alt`, `title`) dynamically at runtime using `setAttribute()` and `removeAttribute()`.
- To implement state reset routines restoring initial element text, layout properties, and styling definitions upon user trigger.

4. THEORY

The Document Object Model (DOM) is a cross-platform, language-neutral Application Programming Interface (API) that represents an HTML document as a logical tree structure. Each branch of the tree terminates in a node, and each node contains objects that can be programmatically accessed and manipulated by JavaScript. This enables modern web pages to be fully dynamic and reactive to user actions.

Node selection is conducted using `document.getElementById()`, which provides efficient $O(1)$ hash map lookup in browser engines. When modifying textual content, the `.textContent` property is favored over `.innerHTML` because it treats input strictly as plain text, preventing Cross-Site Scripting (XSS) injection attacks and eliminating costly HTML parsing overhead.

Dynamic styling is managed through the `element.style` interface. Inline styles set via JavaScript override stylesheet rules due to standard CSS specificity. JavaScript property names adopt camelCase conventions (e.g., `style.backgroundColor` for CSS `background-color`). Adjusting properties such as `borderRadius` ('50%' for circular transformations) or dimensions triggers browser recalculation of geometry (reflow) and visual appearance (repaint) smoothly.

Attribute manipulation is governed by `setAttribute()`, `getAttribute()`, and `removeAttribute()`. Unlike direct object property access, `setAttribute()` updates the literal HTML markup attributes, making it ideal for changing image sources (`src`), alternative text (`alt`), and hover tooltip labels (`title`). A structured reset function demonstrates comprehensive state cleanup by reversing all DOM mutations.

5. ALGORITHM / PROCEDURE

1. Create an HTML file named `dom.html`.
2. Set up the HTML5 boilerplate with meta viewport for responsive scaling and title 'DOM Interactive Demo'.
3. Write internal CSS styling: configure a diagonal background gradient (`linear-gradient(135deg, #667eea, #764ba2)`), centered flexbox layout, and white card container with rounded corners (25px) and elevated box-shadow.
4. Style typography, heading boxes (`#heading`), descriptive paragraph (`#message`), image container (`#image`), interactive buttons with hover transitions, and a live status indicator (`#status`).
5. In the HTML body, assemble the card layout containing heading, paragraph, image placeholder, four action buttons, and status paragraph.
6. Assign distinct onclick event handlers to each button: `changeContent()`, `changeStyle()`, `changeAttribute()`, and `resetPage()`.
7. Define `changeContent()` in `<script>`: select `#heading` and `#message`, update their `textContent` to 'Updated Heading' and 'The paragraph content has been changed.', and update status to 'Content changed'.
8. Define `changeStyle()` in `<script>`: retrieve `#heading` and `#image`; change heading colors (`color: white, backgroundColor: #9333ea`), increase font size (28px), and transform image shape (`borderRadius: 50%, borderColor: #9333ea`); update status to 'Style changed'.
9. Define `changeAttribute()` in `<script>`: retrieve `#image`; invoke `setAttribute()` to update `src` to a new image URL, set `alt` to 'Updated Image', and set `title` to 'Image attribute changed'; update status to 'Attribute changed'.
10. Define `resetPage()` in `<script>`: restore original text content for heading and paragraph; clear custom inline styles for heading and image; restore original image `src` and `alt` attributes; remove title attribute using `removeAttribute('title')`; and set status back to 'Ready'.
11. Open `dom.html` in a web browser and verify interactive execution across each button.
12. Verify smooth transitions, responsive card dimensions, visual feedback, and accurate DOM state restoration.

6. PROGRAM

dom.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>DOM Interactive Demo</title>

  <style>
    * {
      box-sizing: border-box;
    }

    body {
      margin: 0;
      font-family: Arial, sans-serif;

      background: linear-gradient(
        135deg,
        #667eea,
        #764ba2
      );

      min-height: 100vh;

      display: flex;
      justify-content: center;
      align-items: center;

      padding: 25px;
    }

    .container {
      width: 100%;
      max-width: 600px;

      background: white;

      padding: 35px;

      border-radius: 25px;

      text-align: center;

      box-shadow:
        0 15px 40px rgba(0, 0, 0, 0.25);
    }

    h1 {
      color: #4f46e5;
      margin-bottom: 10px;
    }

    .subtitle {
      color: #64748b;
      margin-bottom: 25px;
    }
  </style>
</head>
<body>
  <div class="container">
    <h1>DOM Interactive Demo</h1>
    <div class="subtitle">
      A hands-on guide to manipulating the Document Object Model (DOM)
    </div>
  </div>
</body>
</html>
```

```
#heading {
  color: #7c3aed;

  background: #f3e8ff;

  padding: 18px;

  border-radius: 15px;

  transition: 0.3s;
}

#message {
  color: #475569;

  font-size: 18px;

  line-height: 1.6;

  margin: 20px 0;
}

#image {
  width: 150px;
  height: 150px;

  object-fit: cover;

  border-radius: 20px;

  border: 5px solid #c4b5fd;

  transition: 0.3s;
}

.buttons {
  display: flex;

  flex-wrap: wrap;

  justify-content: center;

  gap: 10px;

  margin-top: 25px;
}

button {
  border: none;

  border-radius: 12px;

  padding: 12px 18px;

  font-size: 14px;

  font-weight: bold;

  color: white;
```

```

        cursor: pointer;

        transition: 0.2s;
    }

    button:hover {
        transform: translateY(-2px);

        opacity: 0.9;
    }

    .content-btn {
        background: #2563eb;
    }

    .style-btn {
        background: #9333ea;
    }

    .attribute-btn {
        background: #059669;
    }

    .reset-btn {
        background: #64748b;
    }

    #status {
        margin-top: 20px;

        color: #64748b;

        font-size: 14px;

        font-weight: bold;
    }
</style>
</head>

<body>

    <div class="container">

        <h1>DOM Interactive Demo</h1>

        <p class="subtitle">
            Explore how JavaScript changes HTML elements.
        </p>

        <h2 id="heading">
            Welcome to the Webpage
        </h2>

        <p id="message">
            This is a sample paragraph.
        </p>
    </div>

```

```


<div class="buttons">

  <button
    class="content-btn"
    onclick="changeContent()">
    Change Content
  </button>

  <button
    class="style-btn"
    onclick="changeStyle()">
    Change Style
  </button>

  <button
    class="attribute-btn"
    onclick="changeAttribute()">
    Change Attribute
  </button>

  <button
    class="reset-btn"
    onclick="resetPage()">
    Reset
  </button>

</div>

<p id="status">
  Ready
</p>

</div>

<script>

  // Change the content of HTML elements
  function changeContent() {

    document.getElementById("heading").textContent =
      "Updated Heading";

    document.getElementById("message").textContent =
      "The paragraph content has been changed.";

    document.getElementById("status").textContent =
      "Content changed";

  }
}
```

```
// Change the style of HTML elements
function changeStyle() {

    let heading =
        document.getElementById("heading");

    let image =
        document.getElementById("image");

    heading.style.color = "ffffff";

    heading.style.backgroundColor = "#9333ea";

    heading.style.fontSize = "28px";

    image.style.borderColor = "#9333ea";

    image.style.borderRadius = "50%";

    document.getElementById("status").textContent =
        "Style changed";

}

// Change HTML attributes
function changeAttribute() {

    let image =
        document.getElementById("image");

    image.setAttribute(
        "src",
        "https://placeholder.co/150x150/06b6d4/ffffff?text=NEW"
    );

    image.setAttribute(
        "alt",
        "Updated Image"
    );

    image.setAttribute(
        "title",
        "Image attribute changed"
    );

    document.getElementById("status").textContent =
        "Attribute changed";

}

// Reset the webpage
```

```
function resetPage() {

    document.getElementById("heading").textContent =
        "Welcome to the Webpage";

    document.getElementById("message").textContent =
        "This is a sample paragraph.";

    let heading =
        document.getElementById("heading");

    heading.style.color = "#7c3aed";

    heading.style.backgroundColor = "#f3e8ff";

    heading.style.fontSize = "";

    let image =
        document.getElementById("image");

    image.setAttribute(
        "src",
        "https://placeholder.co/150x150/7c3aed/ffffff?text=IMAGE"
    );

    image.setAttribute(
        "alt",
        "Sample Image"
    );

    image.removeAttribute("title");

    image.style.borderColor = "#c4b5fd";

    image.style.borderRadius = "20px";

    document.getElementById("status").textContent =
        "Ready";

}

</script>

</body>
</html>
```

7. CODE EXPLANATION

Code	Explanation
<code>document.getElementById("heading").textContent = ...</code>	Replaces the inner plain text of the heading node securely without triggering HTML parsing or XSS vulnerabilities.
<code>heading.style.backgroundColor = "#9333ea";</code>	Dynamically updates the element's inline CSS background color using camelCase syntax, overriding external stylesheet rules.
<code>heading.style.fontSize = "28px";</code>	Dynamically adjusts the typography font size of the heading element at runtime.
<code>image.style.borderRadius = "50%";</code>	Transforms the rectangular image container into a circular avatar by modifying border radius.
<code>image.style.borderColor = "#9333ea";</code>	Dynamically changes the outer border stroke color of the image element.
<code>image.setAttribute("src", "...");</code>	Updates the HTML 'src' attribute of the image, causing the browser to fetch and render a new visual asset.
<code>image.setAttribute("alt", "Updated Image");</code>	Sets the accessible alternative text attribute of the image element.
<code>image.setAttribute("title", "...");</code>	Injects an advisory tooltip attribute that appears when a user hovers the cursor over the image.
<code>image.removeAttribute("title");</code>	Completely removes the 'title' attribute node from the HTML element during the reset cycle.
<code>linear-gradient(135deg, #667eea, #764ba2)</code>	Applies a modern, high-contrast diagonal gradient background to the entire viewport container.

8. TEST CASES AND EXECUTION DATA

Test Case	Button Action	Category	Trigger / Input	Expected Result	Actual Result	Status
TC-01	Page Load / Reset	Default State	Initial load or click 'Reset'	Original text, lavender box, square image, status 'Ready'	Default layout rendered accurately	Pass
TC-02	Change Content	Content Modification	Click 'Change Content' button	Heading: 'Updated Heading', Paragraph updated, status 'Content changed'	Text updated without reload	Pass
TC-03	Change Style	Style Transformation	Click 'Change Style' button	Heading purple background with white text; image becomes circular (50%)	Styles applied via style property	Pass
TC-04	Change Attribute	Attribute Modification	Click 'Change Attribute' button	New cyan image loaded, alt='Updated'	Attributes updated via setAttribute	Pass

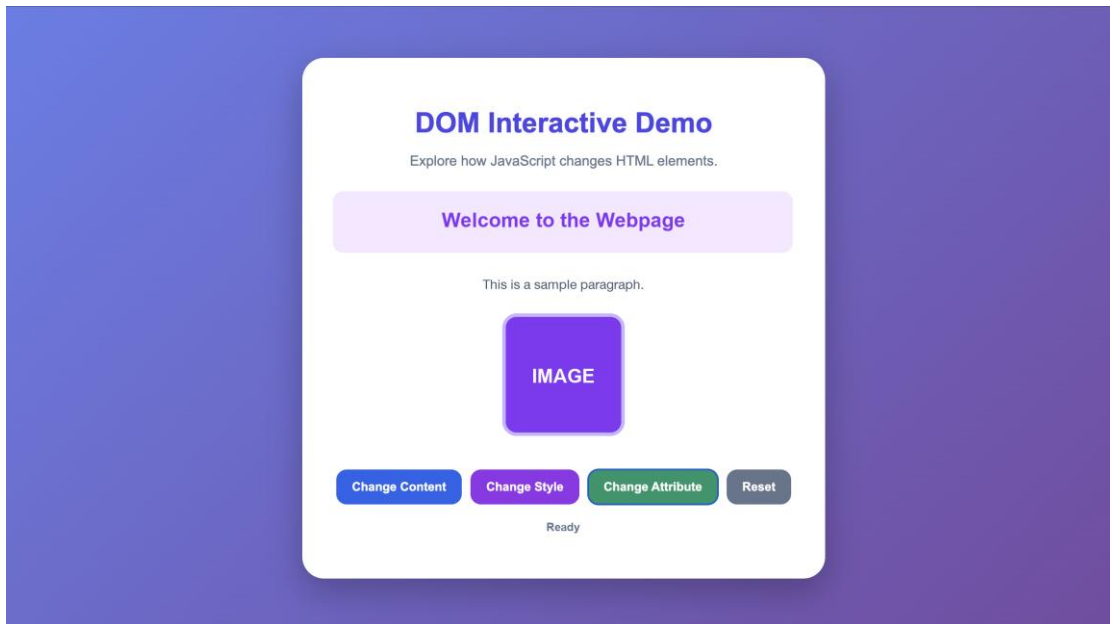
				Image', title tooltip added		
TC-05	Reset Page	Full State Restoration	Click 'Reset' after modifications	All styles, text, image src, and status restored to default state	Complete DOM restoration verified	Pass

9. OUTPUT

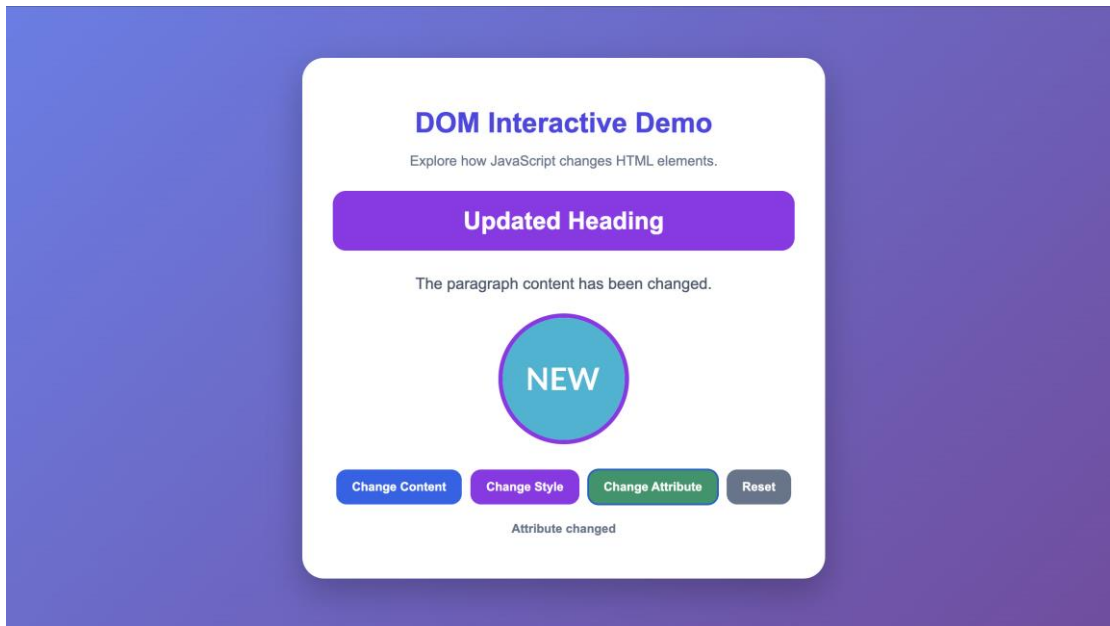
Output 1: DOM Interaction Event Trace

```
[Event 1] Loaded dom.html in browser window.
[Event 2] DOM tree parsed: #heading, #message, #image, #status elements ready.
[Event 3] Default state active: Status = 'Ready', Image = 'IMAGE' (border-radius: 20px).
[Event 4] User clicked 'Change Content' button -> changeContent() invoked.
[Event 5] DOM update: #heading.textContent -> 'Updated Heading'.
[Event 6] DOM update: #message.textContent -> 'The paragraph content has been changed.'.
[Event 7] DOM update: #status.textContent -> 'Content changed'.
[Event 8] User clicked 'Change Style' button -> changeStyle() invoked.
[Event 9] Style update: #heading.style.backgroundColor -> '#9333ea', color -> '#ffffff', fontSize -> '28px'.
[Event 10] Style update: #image.style.borderRadius -> '50%', borderColor -> '#9333ea'.
[Event 11] User clicked 'Change Attribute' button -> changeAttribute() invoked.
[Event 12] Attribute update: #image.setAttribute('src', '...text=NEW'), setAttribute('alt', 'Updated Image').
[Event 13] DOM update: #status.textContent -> 'Attribute changed'.
[Event 14] User clicked 'Reset' button -> resetPage() invoked; all nodes restored to default state.
```

Output 2: Default Initial State Application



Output 3: Dynamic Multi-Attribute & Style Modified State



10. RESULT

Thus, an interactive web application demonstrating comprehensive DOM manipulation across textual content, inline CSS styles, and HTML element attributes with dynamic state reset was successfully designed, implemented, and verified across all test cases.